

Содержание

9 Задача о наибольшей клике	42
9.1 Алгоритмы типа ветвлений	42
9.2 Локальный поиск	43
9.3 Методы, основанные на подграфах	43

9 Задача о наибольшей клике

В этом пункте мы рассмотрим задачу нахождения клики максимальной мощности в графе.

Определение 1. Кликкой в неориентированном графе $G = (V, E)$ называется подмножество $C \subseteq V$ вершин такое, что подграф, индуцированный C — полный, т.е., любые две вершины из C соединены ребром.

Максимальной кликой называется клика, к которой нельзя добавить вершину без разрушения свойства клики.

Наибольшая клика — это клика наибольшей мощности в графе. Её мощность называется кликовым числом и обозначается $\omega(G)$.

Следующий результат очевидный.

Лемма 1. Пусть $C \subseteq V$ — подмножество вершин в графе $G = (V, E)$.

Следующие условия эквивалентны:

- подмножество вершин $C \subseteq V$ является (максимальной, наибольшей) кликой в графе G
- подмножество вершин $C \subseteq V$ является (максимальным, наибольшим) независимым множеством в комплементарном графе $\bar{G}$
- подмножество вершин $V \setminus C$ является (минимальным, наименьшим) вершинным покрытием в комплементарном графе $\bar{G}$

Кликовое число G равно вершинному числу независимости $\bar{G}$. □

Задачей о наибольшей клике (MaxClique) называется проблема нахождения наибольшей клики в данном графе G . Для графов общего вида это НР-сложная задача.

Эта задача эквивалентна задаче о наибольшем независимом множестве (maximum stable set) и задаче о наименьшем вершинном покрытии (minimum vertex cover).

9.1 Алгоритмы типа ветвлений

Алгоритм ветвей и границ строит дерево поиска, в котором узлы индизированы парами (C, W) , где $C \subseteq V$ — клика, а $W \subseteq V$ — подмножество вершин, которые потенциально могут к этой клике присоединиться в дочерних узлах. Данный узел C может породить только такие дочерние узлы (C', W') , в которых $C \subset C'$, $W' \supset W$.

Корневой узел инициализируется парой $(\emptyset, V)$. Ветвление узла (C, W) происходит по вершине $v \in W$. Создаются два дочерних узла, в одном из которых v обязательно присутствует в клике, в другом обязательно отсутствует.

В узле (C, W) из W можно удалить все вершины, которые не смежны хотя бы с одной вершиной из C . Поэтому дочерние узлы узла (C, W) характеризуются парами

$$(C \cup \{v\}, W \setminus (\{v\} \cup \{w \mid (v, w) \notin E\})), (C, W \setminus \{v\}).$$

Для ускорения алгоритма ветвей и границ в задаче на максимум привлекаются верхние оценки на значения функционала для всех потенциальных дочерних узлов данного узла. Для получения верхней оценки на кликовое число индуцированного над $C \cup W$ подграфа можно использовать следующий результат.

Лемма 2. Пусть $G = (V, E)$ — граф, $C \subset V$ — клика. Определим $W = \{w \in V \mid (c, w) \in E \ \forall c \in C\}$. Пусть $C' \supset C$ — большая клика. Тогда $C' \subset C \cup W$ и $|C'| \leq |C| + \chi(W)$, где $\chi(W)$ — хроматическое число индуцированного над W подграфа.

Доказательство. Так как для любых $c \in C$, $v \in C' \setminus C$ имеем $(c, v) \in E$, получаем $C' \setminus C \subset W$. Далее, $C' \setminus C$ также является кликой, и поэтому

$$|C' \setminus C| \leq \omega(W) \leq \chi(W).$$

□

Отсюда следует, что если π — раскраска индуцированного на W подграфа в $\bar{\chi}(W)$ цветов, то $|C| + \bar{\chi}(W)$ является верхней оценкой на мощность клик, которые могут возникнуть в дочерних узлах узла (C, W) . Таким образом, задача раскраски появляется как подзадача для поиска верхних оценок в узлах дерева поиска.

Если в каком-то узле верхняя граница оказывается не больше, чем мощность уже найденной в ходе работы алгоритма клики, то этот узел далее ветвить не надо, так как он заведомо не улучшит уже найденное решение.

Правило выбора вершины $v \in W$ для ветвления можно адаптировать с целью улучшения верхней границы. В частности, можно ветвить по вершинам из маломощных цветовых классов, чтобы поскорее от них избавиться и понизить число цветов в раскраске W . Также можно модифицировать раскраску, перемещая вершины из маломощных цветовых классов в более мощные. Например, можно упорядочить вершины по нисходящей мощности цветовых классов и пройти по графу жадным алгоритмом.

9.2 Локальный поиск

В задаче о максимальной клике преобладают две стратегии локального поиска.

В первой группе на каждом верхнеуровневом шаге ищется клика мощности k , в предположении, что на предыдущем шаге была найдена клика мощности $k - 1$. На низшем уровне

- итерации по подмножествам вершин мощности k
- функционал на минимизацию — число недостающих рёбер между вершинами подмножества
- оператор перехода в близкую точку — замена d вершин в подмножестве

Во второй группе алгоритм поиска ходит по кликам в графе.

- итерации по кликам $C \subset V$
- функционал на максимизацию — мощность клики k
- оператор перехода в близкую точку — замена и / или добавление d вершин в клику

Генетические алгоритмы плохо работают на задаче о наибольшей клике, так как трудно комбинировать родительские клики с целью получить дочернюю.

9.3 Методы, основанные на подграфах

На хордальном графе построить наибольшую клику является простой задачей. Более того, поскольку хордальный граф является совершенным, наибольшая клика имеет мощность, равную его хроматическому числу. Этот факт лежит в основе методов, основанных на поиске хордальных подграфов [1].

Алгоритм сначала находит максимальный хордальный индуцированный подграф G' . Для этого к исходному графу применяется алгоритм поиска совершенного порядка исключения. Напомним, что этот алгоритм назначает вершинам порядковый номер с конца. Можно модифицировать этот алгоритм таким образом, что перед назначением этого номера проверяется, не нарушает ли вершина свойство совершенного порядка исключения. В таком случае вершина отбрасывается. В итоге получаем следующий алгоритм.

создать упорядоченный список L подмножеств V , с инициализацией $L\{1\} = \{1, \dots, n\}$

создать вектор s из n индексов, с любой инициализацией

$i \leftarrow n$

while $i > 0$ **do**

$j \leftarrow L\{1\}(1)$

удалить j из $L\{1\}$

$J \leftarrow \{s(l) \mid l > i, (j, s(l)) \in E\}$

if J — клика **then**

$s(i) \leftarrow j$

$i \leftarrow i - 1$

if $L\{1\} = \emptyset$ **then**

удалить $L\{1\}$, при этом индексы других $L\{k\}$ соответственно уменьшатся на 1

end if

for $k = 1 : |L|$ **do**

разделить $L\{k\}$ на два подмножества, первое из смежных к j вершин, второе из оставшихся
если оба подмножества непустые, заменить $L\{k\}$ на эту пару (список удлинится на 1)

end for

end if

end while

В итоге максимальный хордальный подграф $G' = (V', E')$ состоит из вершин $\{s(l) \mid l > i\}$, и s задаёт порядок совершенного исключения на нём.

Пусть $C \subset V'$ — наибольшая клика в подграфе G' , а π — оптимальная раскраска. Так как G' — хордальный, имеем $|C| = \omega(G') = \chi(G')$.

Далее можно расширять граф G' , не меняя при этом его хроматическое число. Для этого к G' можно присоединять такие вершины из $V \setminus V'$, которые можно покрасить в уже используемые цвета, т.е., которые не смежны всем цветовым классам одновременно. Для расширенного графа $G'' = (V'', E'')$ имеем $\chi(G'') = \chi(G') = |C|$, поэтому клика C всё ещё наибольшая в G'' .

Найденную пару (C, G'') можно, например, использовать в качестве стартовой точки для локального поиска или процесса ветвления. В последнем случае можно отбросить вершины из $V'' \setminus C$ и начать с узла $(C, V \setminus V'')$.

Список литературы

- [1] Egon Balas and Chang Sung Yu. Finding a maximum clique in an arbitrary graph. *SIAM J. Comput.*, 15(4):1054–1068, 1986.